ООП

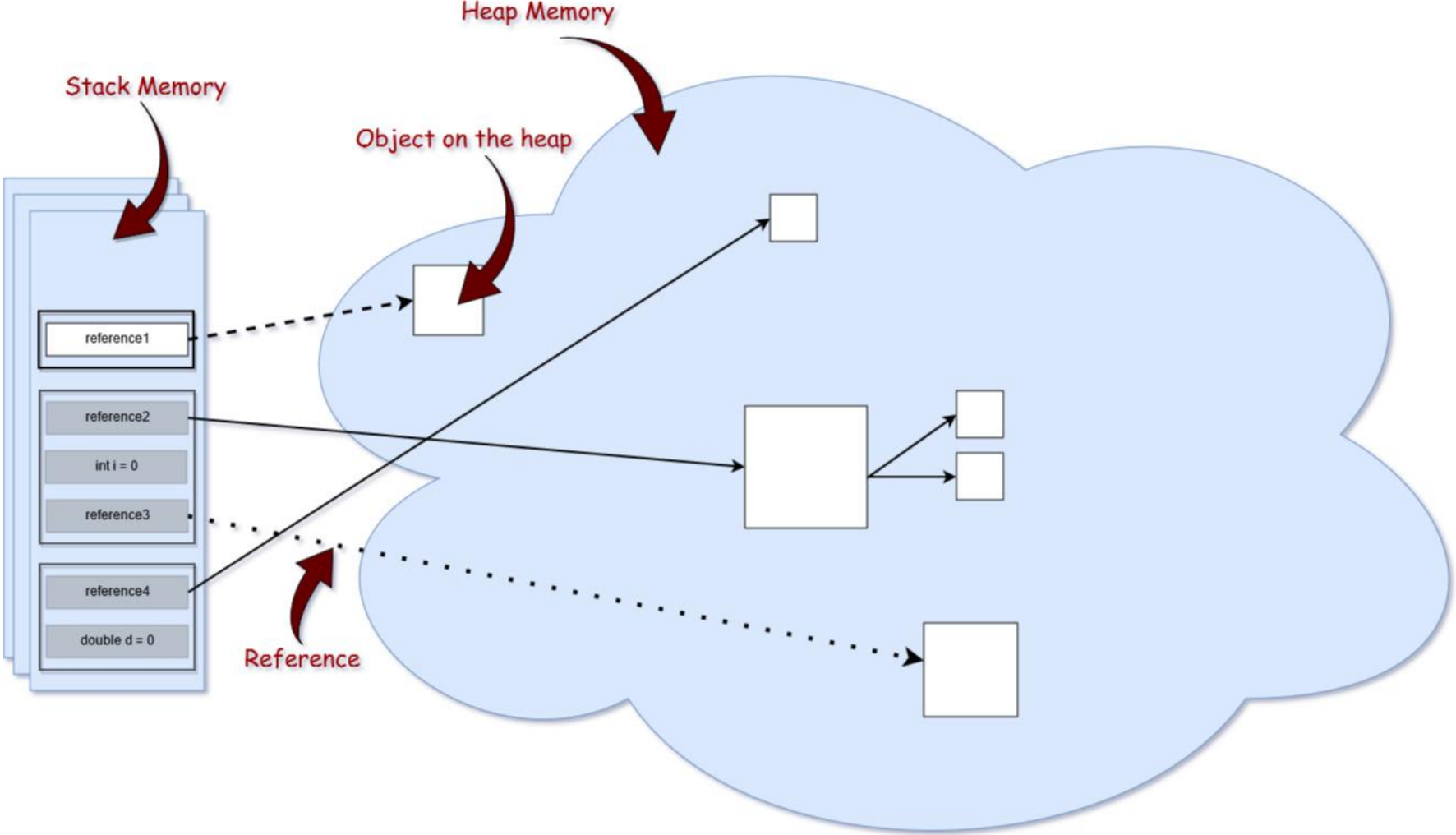
Объектно-ориентированное программирование

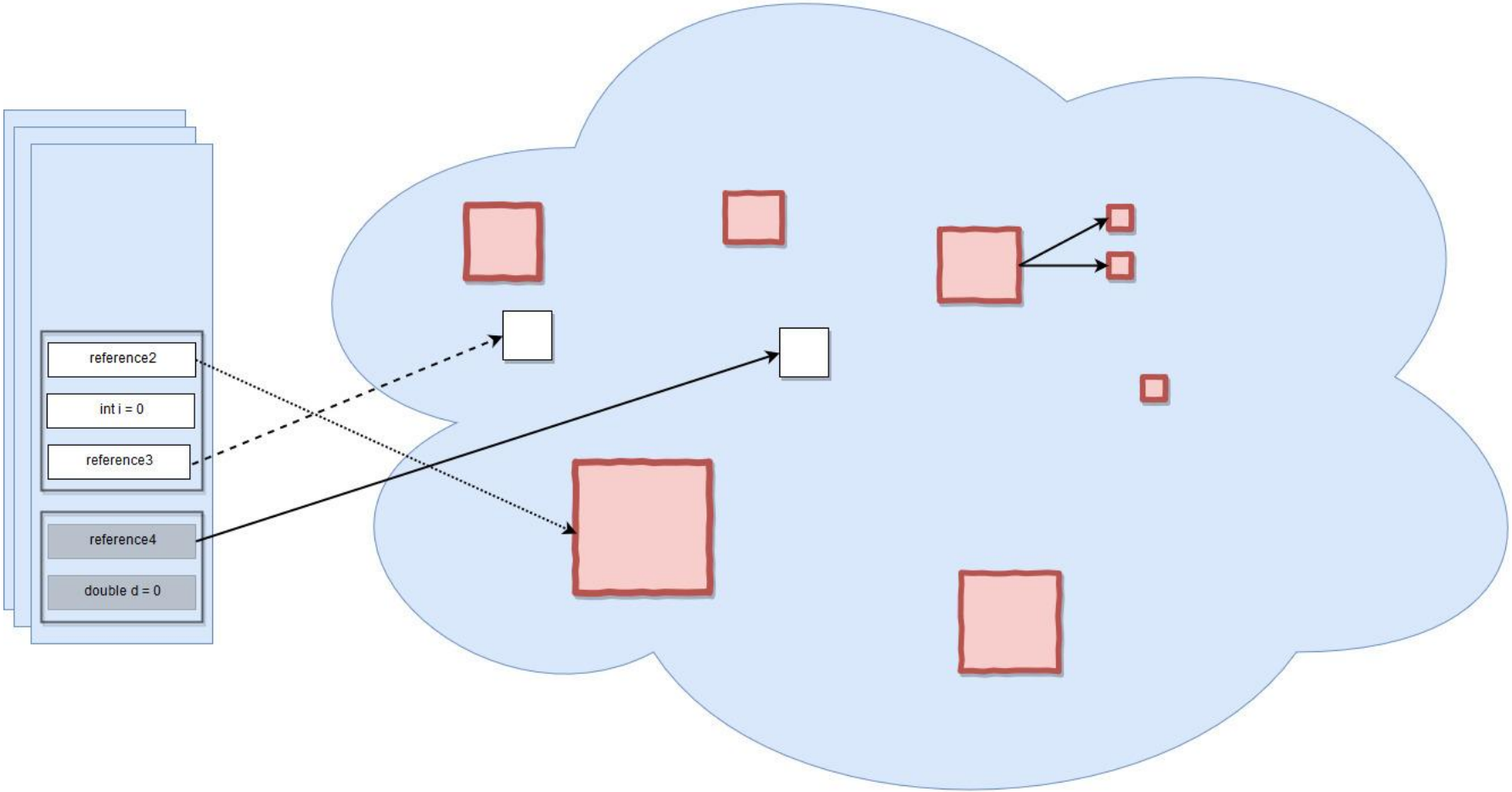
Объектно-ориентированное программирование

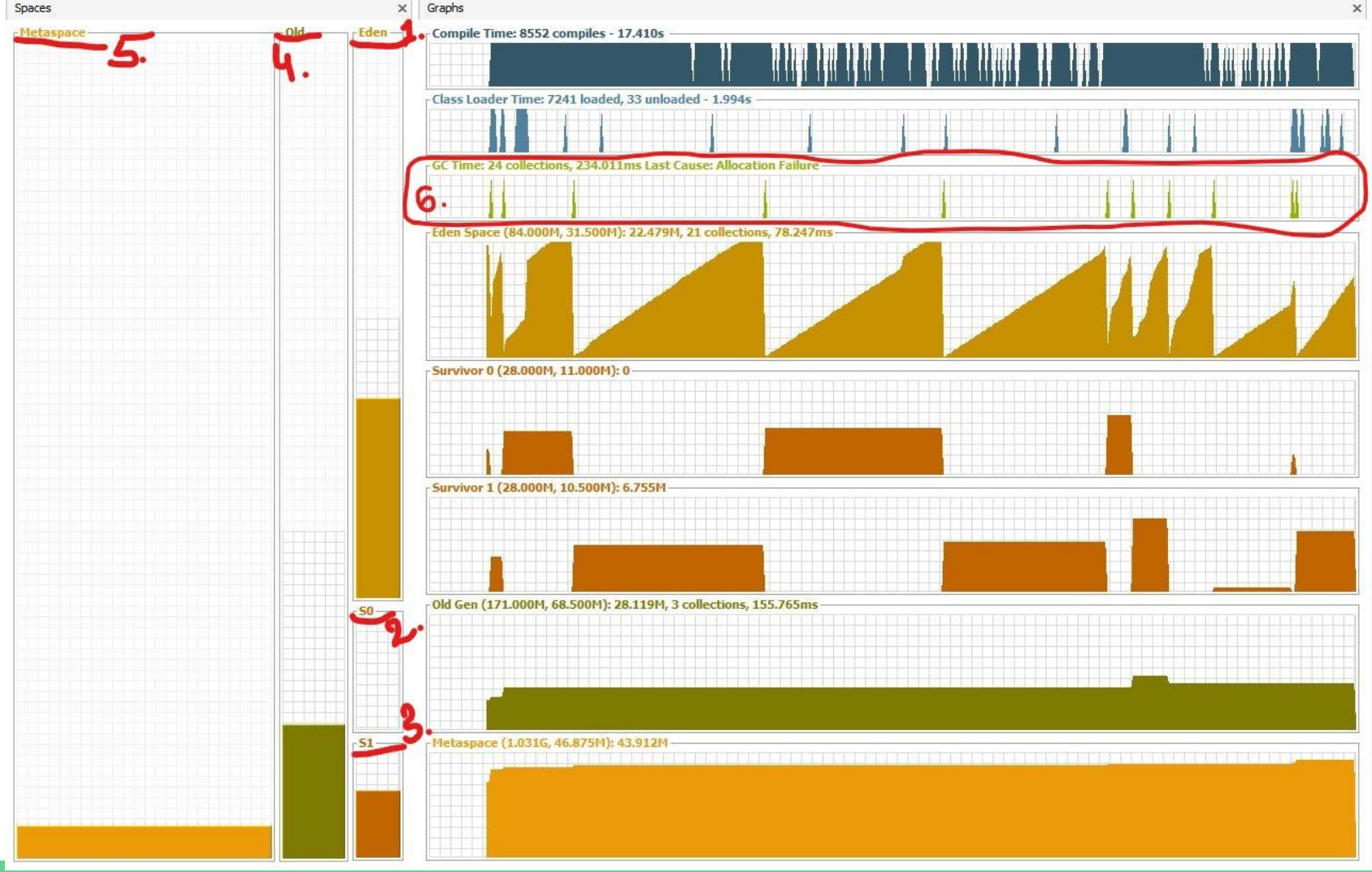
это подход к написанию программ, в котором главным элементом выступают объекты

методология или стиль программирования на основе описания типов/моделей предметной области и их взаимодействия, представленных порождением из прототипов или как экземпляры классов, которые образуют иерархию наследования.

это методология, основанная на представлении программы как совокупности взаимодействующих объектов.


Heap Memory
Stack Memory
Object on the heap
reference1
reference2
int i = 0
reference3
reference4
double d = 0
Reference


reference2
int i = 0
reference3
reference4
double d = 0


Spaces
Graphs
Metaspace
Old
Eden
S0
S1
5.
4.
1.
2.
3.
6.
Compile Time: 8552 compiles - 17.410s
Class Loader Time: 7241 loaded, 33 unloaded - 1.994s
GC Time: 24 collections, 234.011ms Last Cause: Allocation Failure
Eden Space (84.000M, 31.500M): 22.479M, 21 collections, 78.247ms
Survivor 0 (28.000M, 11.000M): 0
Survivor 1 (28.000M, 10.500M): 6.755M
Old Gen (171.000M, 68.500M): 28.119M, 3 collections, 155.765ms
Metaspace (1.031G, 46.875M): 43.912M

Сравнение строк: == против equals()

```
String s1 = new String("Java");

String s2 = new String("Java");


System.out.println(s1 == s2);        // false - разные объекты

System.out.println(s1.equals(s2));   // true - одинаковое содержимое
```

Про управление памятью подробно

https://habr.com/ru/articles/549176/

https://www.freecodecamp.org/news/garbage-collection-in-java-what-is-gc-and-how-it-works-in-the-jvm/

Ссылки и объекты

https://javarush.com/quests/lectures/questsyntaxpro.level07.lecture02

https://javarush.com/groups/posts/2642-ssihlochnihe-tipih-dannihkh-v-java

Объектно-ориентированное программирование

Наследование: новый класс берет поля и методы существующего (родительского), это повторное использование кода.

Инкапсуляция: прячем внутренние данные класса и даем доступ только через открытые методы.

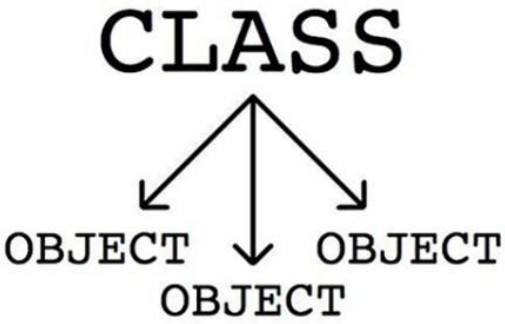
Полиморфизм: с объектами разных типов работаем через общий интерфейс, а поведение зависит от конкретного типа.

(Абстракция): выделяем главные характеристики объекта и отбрасываем несущественные.

Классы

Класс – логическое описание чего-либо, шаблон, с помощью которого можно создавать реальные экземпляры этого самого чего-либо. Другими словами, это просто описание того, какими должны быть созданные сущности: какими свойствами и методами должны обладать.

Свойства – характеристики сущности, методы – действия, которые она может выполнять.



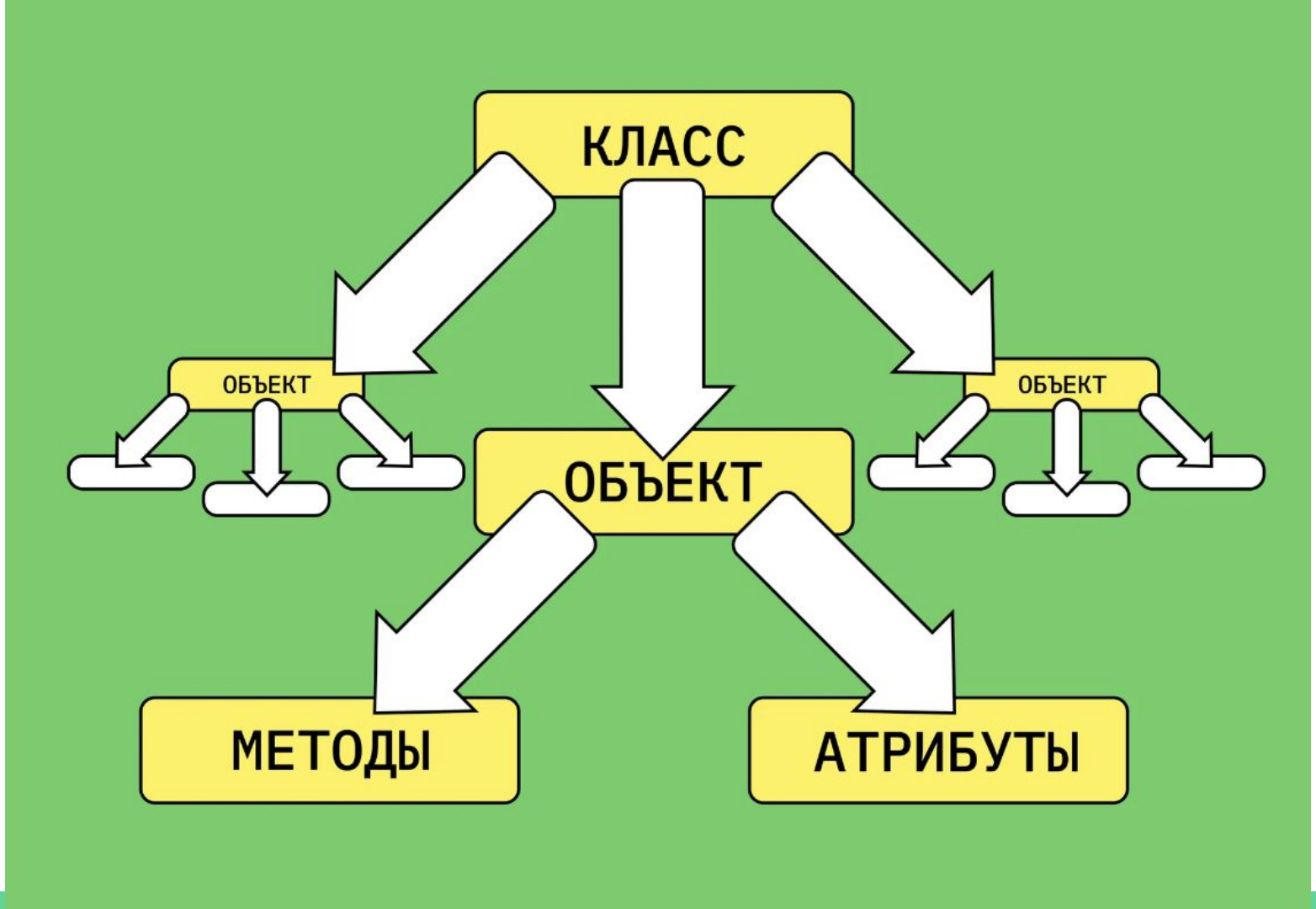
Методы

Функция внутри объекта или класса, которая позволяет взаимодействовать с ним или другой частью кода. В примере с карточками товара метод может:

- Заполнить карточку конкретного объекта нужной информацией.
- Обновлять количество товара в наличии, сверяясь с БД.
- Сравнивать два товара между собой.
- Предлагать купить похожие товары.

Атрибуты (поля)

Характеристики объекта в программировании — например, цена, производитель или объём оперативной памяти. В классе прописывают, что такие атрибуты есть, а в объектах с помощью методов заполняют эти атрибуты данными.


КЛАСС
ОБЪЕКТ
ОБЪЕКТ
ОБЪЕКТ
МЕТОДЫ
АТРИБУТЫ

Классы. Ключевое слово this

Ключевое слово **this** в Java используется для ссылки на текущий объект. Другими словами, **this** ссылается на экземпляр класса, внутри которого оно используется.

- Когда у переменной экземпляра класса и переменной метода/конструктора одинаковые имена;
- Когда нужно вызвать конструктор одного типа (например, конструктор по умолчанию или параметризированный) из другого. Это еще называется явным вызовом конструктора (связный конструктор).

Классы

```
public class Student {
    private String name, group, specialty;

    public Student(String name, String group, String specialty) {
        this.name = name;
        this.group = group;
        this.specialty = specialty;
    }

    // getters/setters
}
```

Классы. Конструктор

Конструкторы в Java представляют собой специальный блок кода, похожий на метод, предназначенный для инициализации полей объекта при его создании. Он вызывается всегда, когда создается новый экземпляр класса.

Существуют 5 базовых типа конструкторов в Java:

- конструктор по умолчанию (default constructor)
- конструктор без аргументов (no-args constructor)
- конструктор с аргументами (parameterized constructor)
- конструктор копирования
- связный конструктор

Дефолтный конструктор

```
import java.time.LocalDateTime;

public class BankAccount {
    String name;
    LocalDateTime opened;
    double balance;

    @Override
    public String toString() {
        return String.format("Имя: %s \nДата создания: %s \nБаланс: %f",
                name, opened.toString(), balance);
    }
}
```

вызов new BankAccount() вызывает созданный нами конструктор без аргументов и неявно возвращает результат в виде ссылки на только что созданный объект.

```
public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        System.out.println(account);
    }
}
```

Конструктор без аргументов

```
import java.time.LocalDateTime;

class BankAccount {
    String name;
    LocalDateTime opened;
    double balance;

    public BankAccount() {
        name = "";
        opened = LocalDateTime.now();
        balance = 0.0d;
    }

    @Override
    public String toString() {
        return String.format("Имя: %s \nДата создания: %s \nБаланс: %f",
                name, opened.toString(), balance);
     }
 }
```

Конструктор с аргументами

```
import java.time.LocalDateTime;

class BankAccount {
    String name;
    LocalDateTime opened;
    double balance;

    public BankAccount() {
        name = "";
        opened = LocalDateTime.now();
        balance = 0.0d;
    }

    public BankAccount(String name, LocalDateTime opened, double balance) {
        this.name = name;
        this.opened = opened;
        this.balance = balance;
    }

    @Override
    public String toString() {
        return String.format("Имя: %s \nДата создания: %s \nБаланс: %f",
                name, opened.toString(), balance);
    }
}
```

Конструктор с аргументами

```
import java.time.LocalDateTime;
import java.time.Month;

public class Main {
    public static void main(String[] args) {
        LocalDateTime opened = LocalDateTime.of(1986, Month.MAY, 30, 4, 30, 0);
        BankAccount account = new BankAccount("Tom", opened, 12_642.0f);

        System.out.println(account);
    }
}
```

Ключевые моменты

- Конструктор вызывается каждый раз при создании объекта
- Имя конструктора должно совпадать с именем класса
- Конструктор не может иметь возвращаемый тип (даже void)
- К конструктору можно применять модификаторы доступа для контроля его вызова
- Способ и порядок размещения конструкторов: они располагаются сразу после полей в порядке, зависящем от принимаемых ими числа аргументов — от меньшего к большему.
- Конструктор может принимать в качестве аргумента как примитивные типы данных, так и объекты
- Класс может содержать любое количество конструкторов
- Компилятор не сгенерирует конструктор по умолчанию, если программист реализовал свой конструктор

Конструктор копирования

Конструктором копирования (copy constructor) называется специальный конструктор, который принимает в качестве аргумента экземпляр того же класса для создания нового объекта на основе переданного.

```
class BankAccount {
        String name;
        LocalDateTime opened;
        double balance;

        public BankAccount(BankAccount account) {
            name = account.name;
            opened = LocalDateTime.now();
            balance = 0.0f;
        }

        public BankAccount(String name, LocalDateTime opened, double balance) {
            this.name = name;
            this.opened = opened;
            this.balance = balance;
        }

        @Override
        public String toString() {
            return String.format("Имя: %s \nДата создания: %s \nБаланс: %f ",
                name, opened.toString(), balance);
        }
}
```

```
LocalDateTime opened = LocalDateTime.of(1986, Month.MAY, 30, 4, 30, 0);
BankAccount account = new BankAccount("Tom", opened, 12_642.0f);
BankAccount newAccount = new BankAccount(account);
```

Связанные конструкторы

Нужно отметить, что мы не обязаны в конструктор передавать для инициализации всех полей класса какие-то значения. Часть обязанностей можно возложить на компилятор, который присвоит полям значения по умолчанию (если это не критично). Также, в процессе работы программы, можно использовать сеттеры.

Связанные конструкторы

```
class BankAccount {
    String name;
    LocalDateTime opened;
    double balance;

    public BankAccount() {
        this("", LocalDateTime.now(), 0.0d);
    }

    public BankAccount(BankAccount account) {
        this(account.name, LocalDateTime.now(), 0.0f);
    }

    public BankAccount(String name) {
        this(name, LocalDateTime.now(), 0.0f);
    }

    public BankAccount(String name, LocalDateTime opened, double balance) {
        this.name = name;
        this.opened = opened;
        this.balance = balance;
    }
```

```java
public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("Tom");
        System.out.println(account);
    }
}
```

Инициализаторы

Кроме конструктора начальную инициализацию объекта вполне можно было проводить с помощью инициализатора объекта. Инициализатор выполняется до любого конструктора.

```
/*начало блока инициализатора*/
{
    name = "Undefined";
    age = 18;
}
/*конец блока инициализатора*/
```

Модификаторы доступа

public: публичный, общедоступный класс или член класса. Поля и методы, объявленные с модификатором public, видны другим классам из текущего пакета и из внешних пакетов.

private: закрытый класс или член класса, противоположность модификатору public. Закрытый класс или член класса доступен только из кода в том же классе.

protected: такой класс или член класса доступен из любого места в текущем классе или пакете или в производных классах, даже если они находятся в других пакетах

Модификатор по умолчанию. Отсутствие модификатора у поля или метода класса предполагает применение к нему модификатора по умолчанию. Такие поля или методы видны всем классам в текущем пакете.

Инкапсуляция

Изначальное значение слова **«инкапсуляция»** в программировании — объединение данных и методов работы с этими данными в одной упаковке («капсуле»).

В Java в роли упаковки-капсулы выступает класс. Класс содержит в себе и данные (поля класса), и методы для работы с этими данными.

Суть **сокрытия** - все «внутренности» программы скрываются от пользователя. Для него эта информация является лишней, ненужной. Пользователю необходим конечный результат, а не внутренний процесс.

Инкапсуляция

Инкапсуляция — это процесс сокрытия внутренних деталей реализации объекта и предоставления внешнему миру только необходимых методов для взаимодействия с объектом. Это помогает упростить код, предотвращает нежелательное изменение состояния объекта и улучшает безопасность.

Пример инкапсуляции в Java

```
class Person {
    private String name;
    private int age;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

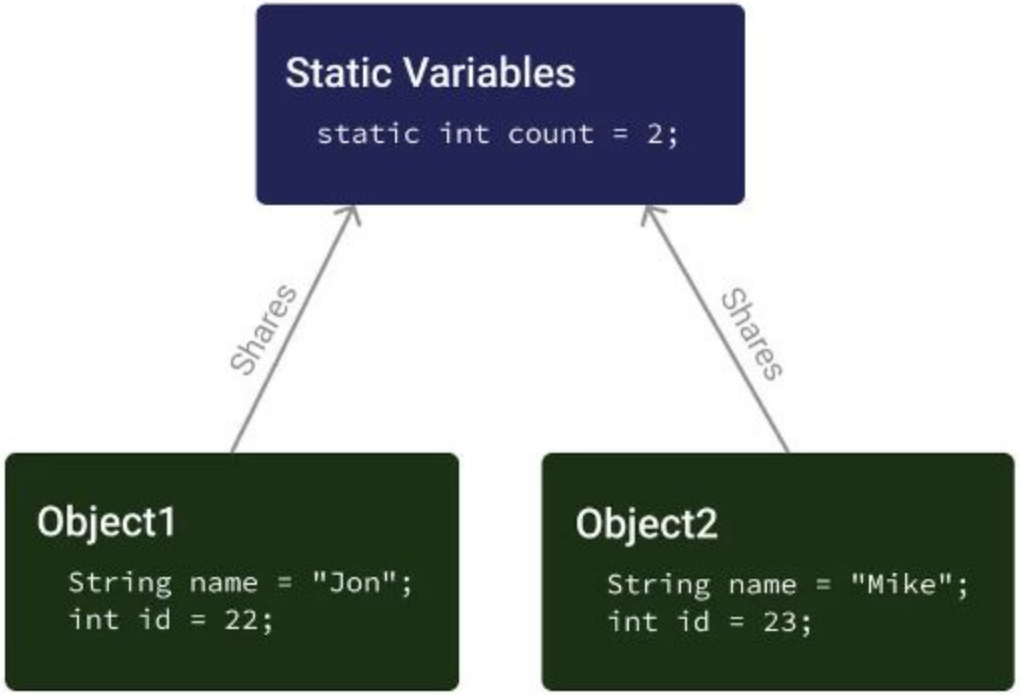
    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age >= 0) {
            this.age = age;
        }
    }
}
```

Static

В языке Java, если поле объявляется статическим (путем добавления модификатора static), то в независимости от количества созданных объектов класса — всегда будет существовать только один экземпляр статического поля. Значение такого поля будет единым и общим для всех объектов класса, содержащих это поле.

Static


Static Variables
static int count = 2;
Shares
Shares
Object1
String name = "Jon";
int id = 22;
Object2
String name = "Mike";
int id = 23;

Пример статического поля

```
public class Car {

    static int numberOfCars;

    private String name;
    private String engine;

    public Car(String name, String engine) {
        this.name = name;
        this.engine = engine;
        numberOfCars++;
    }

    // getters and setters
}
```